

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO THỰC TẬP CƠ SỞ

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG QUẢN LÝ BUG

Tên lớp:

D23CQCE03-B

Giảng viên hướng dẫn:

Nguyễn Thị Thanh Thủy

Nhóm BTL:

05

Danh sách thành viên:

1. Nguyễn Quý Tuấn - B23DCVT440
2. Nguyễn Lê Hoàng – B23DCVT172
3. Nguyễn Duy Nam - B23DCVT295

Hà Nội - 2026

Lời cảm ơn

Nhóm em xin được gửi lời cảm ơn chân thành nhất đến cô Nguyễn Thị Thanh Thủy.

Trong quá trình học tập và tìm hiểu môn Thực tập cơ sở, nhóm em đã nhận được rất nhiều sự quan tâm, giúp đỡ, hướng dẫn tâm huyết và tận tình của cô.

Cô đã giúp nhóm em tích lũy thêm nhiều kiến thức về môn học này để có thể hoàn thành được đề tài: Xây dựng ứng dụng web quản lý và theo dõi lỗi (Bug Tracking System).

Trong quá trình thực hiện bài làm chắc chắn khó tránh khỏi những thiếu sót. Do đó, nhóm em kính mong nhận được những lời góp ý quý báu của cô để bài tập lớn của nhóm em ngày càng hoàn thiện hơn.

Nhóm em xin chân thành cảm ơn!

MỤC LỤC

Lời cảm ơn

I. Khảo sát, xác định chức năng của hệ thống.....	3
1. Tổng quan về đề tài.....	3
2. Các chức năng của hệ thống.....	3
3. Công nghệ sử dụng.....	5
II. Phân tích hệ thống.....	6
1. Mô tả hệ thống bằng ngôn ngữ tự nhiên.....	6
2. Phân tích hệ thống.....	6
III. Thiết kế cơ sở dữ liệu.....	10
1. Lược đồ Thực thể - Liên kết (ER Diagram).....	10
2. Chuẩn hóa lược đồ (Normalization).....	13
3. Lược đồ quan hệ (Relational Schema).....	13
4. Từ điển dữ liệu (Data Dictionary).....	14
IV. Thiết kế giao diện người dùng.....	21
1. Giao diện đăng nhập/đăng ký.....	21
2. Giao diện trang chủ.....	22
3. Giao diện danh sách Bug.....	23
4. Giao diện chi tiết Bug.....	23
5. Giao diện quản trị thành viên.....	24
6. Giao diện nhật ký hoạt động.....	25
V: Cài đặt.....	25
1. Môi trường cài đặt và công nghệ sử dụng.....	25
2. Tổ chức các lớp của chương trình.....	26
VI: Kết Luận và hướng phát triển.....	34
1. Kết quả đạt được.....	34
2. Những điểm còn hạn chế.....	35
3. Hướng phát triển của đề tài.....	36

I. Khảo sát, xác định chức năng của hệ thống

1. Tổng quan về đề tài

a) Lý do chọn đề tài: Trong quy trình phát triển phần mềm (Software Development Life Cycle - SDLC), việc phát sinh lỗi (bug) là điều không thể tránh khỏi. Khi quy mô dự án ngày càng mở rộng và số lượng thành viên tham gia (Developer, Tester, Manager) đông đảo hơn, việc quản lý tiến độ xử lý lỗi bằng các phương pháp thủ công (như dùng file Excel, nhắn tin qua các ứng dụng chat) bộc lộ nhiều hạn chế: khó theo dõi trạng thái, dễ bị trôi thông tin, khó phân quyền người xử lý.

Xuất phát từ thực tế đó, đề tài **XÂY DỰNG HỆ THỐNG QUẢN LÝ LỖI** được lựa chọn nhằm tạo ra một công cụ tập trung giúp các nhóm phát triển phần mềm tối ưu hóa quy trình ghi nhận, theo dõi và khắc phục lỗi. Hệ thống giúp minh bạch hóa tiến độ công việc, đảm bảo mọi lỗi đều được xử lý đúng người, đúng thời điểm, góp phần nâng cao chất lượng sản phẩm phần mềm.

b) Mục tiêu: Xây dựng thành công một ứng dụng nền web (Web-based application) hoạt động ổn định với các mục tiêu cụ thể:

- Về mặt nghiệp vụ: Số hóa quy trình quản lý lỗi. Cho phép tạo dự án (Workspace), ghi nhận lỗi mới, phân công người xử lý (Assignee), quản lý vòng đời của lỗi (từ lúc Mở đến lúc Đóng) thông qua bảng Dashboard trực quan.
- Về mặt kỹ thuật: Áp dụng được các công nghệ phát triển web hiện đại. Xây dựng Backend RESTful API bằng Java Spring Boot, áp dụng bảo mật JWT (JSON Web Token), kết hợp với giao diện Frontend tương tác động bằng HTML/CSS/JS và cơ sở dữ liệu MySQL.

2. Các chức năng của hệ thống

Nhóm chức năng Quản lý Tài khoản và Phân quyền:

- Đăng ký & Đăng nhập: Người dùng có thể tạo tài khoản mới với vai trò mong muốn. Hệ thống sử dụng cơ chế bảo mật JWT để xác thực phiên đăng nhập an toàn.
- Giới hạn quyền hạn dựa trên Role (Phân quyền):
 - + Manager (Quản lý dự án) tạo lỗi, có quyền phân công người xử lý (Assignee) lỗi và xóa lỗi khỏi hệ thống.
 - + Quản trị viên hệ thống (ADMIN) có quyền cao nhất: xem được toàn bộ Workspace của mọi người dùng, can thiệp (xóa bug, xóa thành viên) vào bất kỳ Workspace nào, xem danh sách toàn bộ người dùng và xóa tài khoản khỏi hệ thống.
 - + Developer và Tester trong dự án được quyền xem, tạo lỗi, bình luận, đính kèm tệp tin và cập nhật trạng thái vòng đời lỗi.

Nhóm chức năng Quản lý Không gian làm việc (Workspace):

- Tạo Workspace: Mọi người dùng đã đăng nhập hệ thống đều có thể tạo Workspace mới. Người tạo sẽ được hệ thống tự động thiết lập vai trò Manager của Workspace đó.
- Quản lý thành viên (Chỉ dành cho Manager của Workspace):
 - + Thêm thành viên: Tìm kiếm và thêm người dùng vào Workspace bằng Email.
 - + Xóa thành viên: Loại bỏ thành viên ra khỏi Workspace.
 - + Cập nhật vai trò: Manager trong Workspace có thể thay đổi vai trò của thành viên trong Workspace (ví dụ: chuyển từ Developer sang Tester). Để bảo đảm dự án luôn có người quản lý, hệ thống không cho phép Manager tự thay đổi vai trò của chính mình.
 - + Xóa Workspace: Chỉ Manager trong Workspace mới có quyền xóa hoàn toàn Workspace.

Nhóm chức năng Quản lý Lỗi (Bug Tracking):

- Tạo Lỗi (Create Bug): Tester hoặc các thành viên có thể báo cáo lỗi mới với đầy đủ thông tin: Tiêu đề, Mô tả chi tiết, Mức độ ưu tiên (Low, Medium, High, Critical).

- Bảng điều khiển Kanban (Kanban Board): Hiển thị danh sách lỗi trực quan dưới dạng các cột trạng thái: Open (Mở) → In Progress (Đang xử lý) → Resolved (Đã giải quyết) → Closed (Đã đóng). Người dùng có thể cập nhật trạng thái lỗi.
- Manager quản lý việc giao việc bằng cách chỉ định (Assign) lỗi cho một Developer cụ thể xử lý.
- Bình luận (Comment): Các thành viên có thể trao đổi, thảo luận trực tiếp bên dưới mỗi lỗi.
- Đính kèm tệp (Upload File): Cho phép đính kèm hình ảnh minh chứng (screenshot) hoặc file log để làm rõ nguyên nhân lỗi; các thành viên có thể tải về (download) các tệp đính kèm này.

Nhóm chức năng Dashboard

- Hiển thị màn hình tổng hợp số liệu cho người quản lý.
- Trực quan hóa dữ liệu về số lượng lỗi theo từng trạng thái (Status), số lượng lỗi theo mức độ ưu tiên (Priority), và thống kê khối lượng công việc (số bug đang gán) của từng Developer trong dự án.

3. Công nghệ sử dụng

Dự án được phát triển theo kiến trúc Client-Server với các công nghệ cụ thể như sau:

2.1. Phía Backend (Xử lý máy chủ)

- Ngôn ngữ: Java 17 (Cung cấp hiệu năng và các tính năng hướng đối tượng).
- Framework: Spring Boot 3.5 (Hỗ trợ cấu hình tự động, tăng tốc độ phát triển API).
- Quản lý dự án: Maven (Quản lý các thư viện phụ thuộc và build dự án).
- Bảo mật: Spring Security & JWT (Mã hóa mật khẩu bằng BCrypt và xác thực người dùng qua token JWT).
- Tài liệu API: Swagger UI (SpringDoc OpenAPI)

2.2. Hệ quản trị Cơ sở dữ liệu (Database)

- Hệ quản trị CSDL:MySQL (Hệ cơ sở dữ liệu quan hệ mã nguồn mở, lưu trữ thông tin về người dùng, dự án và lỗi).

2.3. Phía Frontend (Giao diện người dùng)

- Ngôn ngữ:HTML5, CSS3, Vanilla JavaScript (Thiết kế cấu trúc, giao diện Dark Mode và gọi API bất đồng bộ bằng Fetch API).
- Framework UI:Bootstrap 5.3 & Bootstrap Icons 1.11 (Hỗ trợ thiết kế giao diện responsive tương thích mọi thiết bị).

II. Phân tích hệ thống

1. Mô tả hệ thống bằng ngôn ngữ tự nhiên

Hệ thống quản lý lỗi và dự án (Bug Tracking System) này cho phép các nhóm phát triển phần mềm tạo lập, theo dõi và quản lý vòng đời của các lỗi (bug) phát sinh trong quá trình xây dựng dự án.

Người dùng có thể đăng ký tài khoản, tham gia vào các không gian làm việc (Workspace) khác nhau, báo cáo lỗi mới, tương tác thảo luận thông qua bình luận, đính kèm tệp tin và cập nhật trạng thái lỗi trực quan qua bảng Kanban.

Hệ thống cũng cung cấp chức năng quản trị cho Trưởng dự án (Manager), cho phép họ quản lý thành viên, phân công xử lý lỗi cho các lập trình viên và giám sát tiến độ dự án thông qua màn hình thống kê Dashboard.

2. Phân tích hệ thống

a) Chức năng chính

Hệ thống được chia thành 4 nhóm chức năng cốt lõi sau:

1. Quản lý Tài khoản & Phân quyền:

- Đăng ký tài khoản mới và đăng nhập xác thực bằng JWT.
- Phân quyền hai cấp: vai trò hệ thống (ADMIN quản trị toàn cục, USER người dùng thông thường) và vai trò theo từng dự án (Manager, Developer, Tester).

2. Quản lý Không gian làm việc (Workspace):

- Cho phép người dùng tạo mới hoặc xóa Workspace (dự án).
- Quản lý thành viên trong Workspace (thêm thành viên bằng Email, xóa thành viên, thay đổi vai trò thành viên).

3. Quản lý Lỗi (Bug Tracking):

- Tạo mới thông tin lỗi bao gồm: Tiêu đề, mô tả chi tiết, thiết lập mức độ ưu tiên (Low, Medium, High, Critical), gắn nhãn phân loại và chỉ định người xử lý (Assignee).
- Theo dõi tiến độ và cập nhật trạng thái lỗi (Open, In Progress, Resolved, Closed) một cách trực quan trên bảng Kanban thông qua cửa sổ cập nhật thông tin chi tiết.
- Tự động ghi nhận nhật ký lịch sử thay đổi trạng thái của Bug (Ai thay đổi, từ trạng thái nào sang trạng thái nào, thời gian cập nhật).
- Hỗ trợ tương tác nhóm qua mục bình luận (Comment) và tải lên các tệp tin hình ảnh/tài liệu minh chứng lỗi (Attachment) với dung lượng giới hạn dưới 5MB.

4. Thống kê dự án (Dashboard):

- Tổng hợp dữ liệu của dự án bao gồm: số lượng lỗi theo từng trạng thái cụ thể, số lượng lỗi phân bổ theo mức độ ưu tiên và số lượng lỗi được giao cho mỗi lập trình viên đảm nhận.

b) Diễn viên (actor)

Hệ thống bao gồm 4 tác nhân chính tương tác với ứng dụng:

1. Người mới:

- Thực hiện đăng ký tài khoản mới (mặc định là người dùng hệ thống) và đăng nhập vào hệ thống.
- Đăng nhập vào hệ thống.

2. Trưởng dự án (Manager):

- Tạo và quản lý không gian làm việc (Workspace).
- Mời thành viên, xóa thành viên hoặc phân quyền vai trò cho thành viên khác trong dự án.
- Gán (Assign) lỗi cho lập trình viên xử lý.
- Tạo lỗi, cập nhật trạng thái lỗi, xóa lỗi, bình luận, đính kèm file và xem thống kê báo cáo (Dashboard).

3. Lập trình viên (Developer):

- Tham gia dự án khi được mời.
- Là thành viên trực tiếp sửa lỗi trong dự án
- Xem danh sách lỗi được phân công.
- Tiến hành xử lý và cập nhật tiến độ lỗi (chuyển trạng thái lỗi trên bảng Kanban).
- Bình luận trao đổi và đính kèm tệp tin.
- Tạo lỗi

4. Kiểm thử viên (Tester):

- Tham gia dự án khi được mời.
- Là thành viên chịu trách nhiệm kiểm thử chất lượng sản phẩm và phát hiện lỗi
- Phát hiện và tạo báo cáo lỗi (Bug) mới lên hệ thống.
- Kiểm tra chất lượng và theo dõi trạng thái lỗi sau khi Developer sửa xong để đóng lỗi.
- Bình luận trao đổi và đính kèm tệp tin.
- Tạo lỗi

5. Về mặt phân quyền trong hệ thống quản lý Workspace và Bug:
- Quyền quản lý Workspace và Thành viên: Chỉ người dùng giữ vai trò Manager của Workspace mới có toàn quyền quản trị, bao gồm tạo, xóa Workspace, cũng như thêm mới, loại bỏ hoặc thay đổi vai trò của các thành viên khác trong không gian làm việc đó. Các vai trò khác như Developer và Tester không được phép thực hiện các thao tác này.
 - Quyền tương tác và báo cáo lỗi (Bug): Mọi thành viên trong Workspace (bao gồm Manager, Developer, và Tester) đều có quyền xem danh sách lỗi, tạo mới Bug (ở trạng thái chưa gán người xử lý), viết bình luận và tải lên tệp tin đính kèm.
 - Quyền phân công và xử lý lỗi: Chỉ vai trò Manager của Workspace mới có quyền chỉ định, gán hoặc thay đổi người xử lý (Assignee) cho từng Bug. Đồng thời, quyền xóa hoàn toàn một Bug khỏi hệ thống cũng được giới hạn duy nhất cho vai trò Manager để đảm bảo tính toàn vẹn của dữ liệu dự án.

c) Mô tả nghiệp vụ của Trưởng dự án (Manager)

Quy trình làm việc (workflow) của Manager trên hệ thống diễn ra qua các bước chính sau:

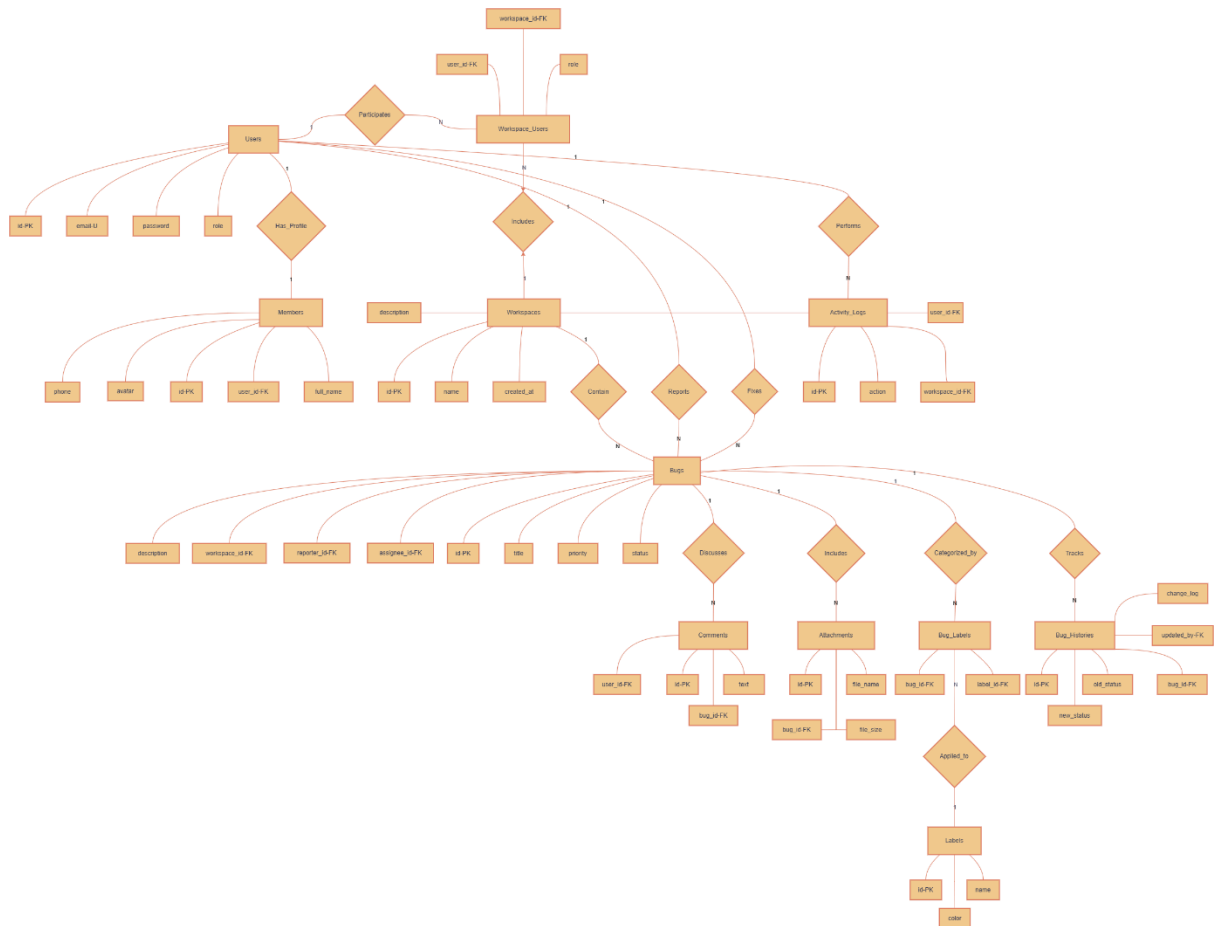
1. Thiết lập dự án: Đăng nhập vào hệ thống và tạo một Không gian làm việc (Workspace) mới cho dự án của mình.
2. Xây dựng đội ngũ:
 - Sử dụng tính năng mời thành viên qua Email để đưa các Developer và Tester vào Workspace.
 - Phân quyền vai trò (Developer hoặc Tester) cho từng thành viên trong Workspace.
3. Điều phối và Giao việc:
 - Kiểm tra các báo cáo lỗi (Bug) mới do các thành viên (thường là Tester) tạo ra trong Workspace.

- Chỉ định (Assign) lập trình viên (Developer) phù hợp chịu trách nhiệm sửa lỗi đó.
 - Xóa bỏ các báo cáo lỗi không hợp lệ hoặc lỗi trùng lặp (Delete Bug).
4. Theo dõi và Đánh giá:
- Quan sát bảng Kanban, là toàn bộ không gian làm việc hiển thị trên giao diện để nắm bắt tiến độ sửa lỗi của nhóm.
 - Truy cập màn hình Dashboard để xem các biểu đồ thống kê trực quan về tổng số lỗi, trạng thái lỗi, mức độ ưu tiên của lỗi và hiệu suất sửa lỗi của từng Developer.

III. Thiết kế cơ sở dữ liệu

1. Lược đồ Thực thể - Liên kết (ER Diagram)

Mô hình ERD thể hiện cấu trúc logic và các ràng buộc nghiệp vụ của hệ thống.



3.1.1. Danh sách các thực thể và thuộc tính chi tiết

Hệ thống bao gồm 11 thực thể chính:

1. **Users (Người dùng):** id, email, password, created_at. Lưu thông tin tài khoản đăng nhập (Email, mật khẩu đã mã hóa).
2. **Members (Hồ sơ):** id, user_id, full_name, avatar, phone, role. Lưu thông tin chi tiết của người dùng (Họ tên, ảnh đại diện - avatar, số điện thoại).
3. **Workspaces (Dự án):** id, name, description, created_at. Lưu thông tin các dự án/không gian làm việc.
4. **Workspace_Users (Thành viên dự án):** workspace_id, user_id, role. Bảng trung gian liên kết người dùng vào workspace, định nghĩa vai trò (role) của người đó trong dự án đó: Manager, Developer, Tester.

5. **Bugs (Lỗi):** id , workspace_id , title, description, priority, status, reporter_id , assignee_id , created_at, updated_at. Lưu thông tin chi tiết về lỗi (Tiêu đề, mô tả, mức độ ưu tiên: Low, Medium, High, Critical; trạng thái: Open, In Progress, Resolved, Closed; người báo cáo, người xử lý).
6. **Labels (Nhãn):** id, name, color. Lưu thông tin các nhãn phân loại kèm mã màu riêng.
7. **Bug_Labels (Gắn nhãn):** bug_id, label_id. Bảng trung gian liên kết các nhãn với các bug (Một bug có thể có nhiều nhãn).
8. **Comments (Bình luận):** id, bug_id, user_id, text, created_at. Lưu các thảo luận, bình luận của các thành viên trực tiếp dưới mỗi bug.
9. **Attachments (Tập đính kèm):** id, bug_id, file_name, uploaded_at, file_size. Quản lý các tập đính kèm được tải lên liên quan đến lỗi (Tên file, kích thước, thời gian tải lên).
10. **Bug_Histories (Lịch sử trạng thái):** id, bug_id, old_status, new_status, updated_by, change_log, created_at. Ghi nhận lịch sử thay đổi trạng thái và thông tin của bug (Ai thay đổi, trạng thái cũ, trạng thái mới, nội dung thay đổi).
11. **Activity_Logs (Nhật ký):** id, user_id, action, created_at. Ghi nhận nhật ký hoạt động chung của người dùng trên hệ thống để phục vụ quản trị bảo mật.

3.1.2. Đặc tả các mối quan hệ (Relationships)

- **Users - Members (1:1):** Mỗi tài khoản có duy nhất một hồ sơ cá nhân.
- **Users - Workspace_Users (1:N):** Một người dùng tham gia vào nhiều dự án khác nhau
- **Workspaces - Workspace_Users (1:N):** Một dự án bao gồm nhiều thành viên.
- **Workspaces - Bugs (1:N):** Một dự án chứa nhiều lỗi; một lỗi chỉ thuộc về một dự án duy nhất.
- **Workspaces - Activity_Logs (1:N):** Một dự án có ghi lại nhiều lịch sử hành động
- **Users - Bugs (1:N) (vai trò reporter):** Một người dùng có thể báo cáo nhiều lỗi (nói qua reporter_id).

- **Users - Bugs (1:N) (vai trò assignee):** Một người dùng có thể xử lý nhiều lỗi (nối qua assignee_id).
- **Bugs - Comments (1:N):** Một lỗi có nhiều bình luận thảo luận.
- **Bugs - Attachments (1:N):** Một lỗi đính kèm nhiều bằng chứng (hình ảnh, tệp tin).
- **Bugs - Bug_Histories (1:N):** Một lỗi ghi lại mọi biến động về trạng thái.
- **Bugs - Bug_Labels (1:N):** Một lỗi có thể được gắn nhiều nhãn dán thông qua bảng trung gian.
- **Labels - Bug_Labels (1:N):** Nhãn dán được áp dụng cho nhiều lỗi thông qua bảng trung gian.
- **Users - Activity_Logs (1:N):** Ghi vết mọi hành động của người dùng trên hệ thống.

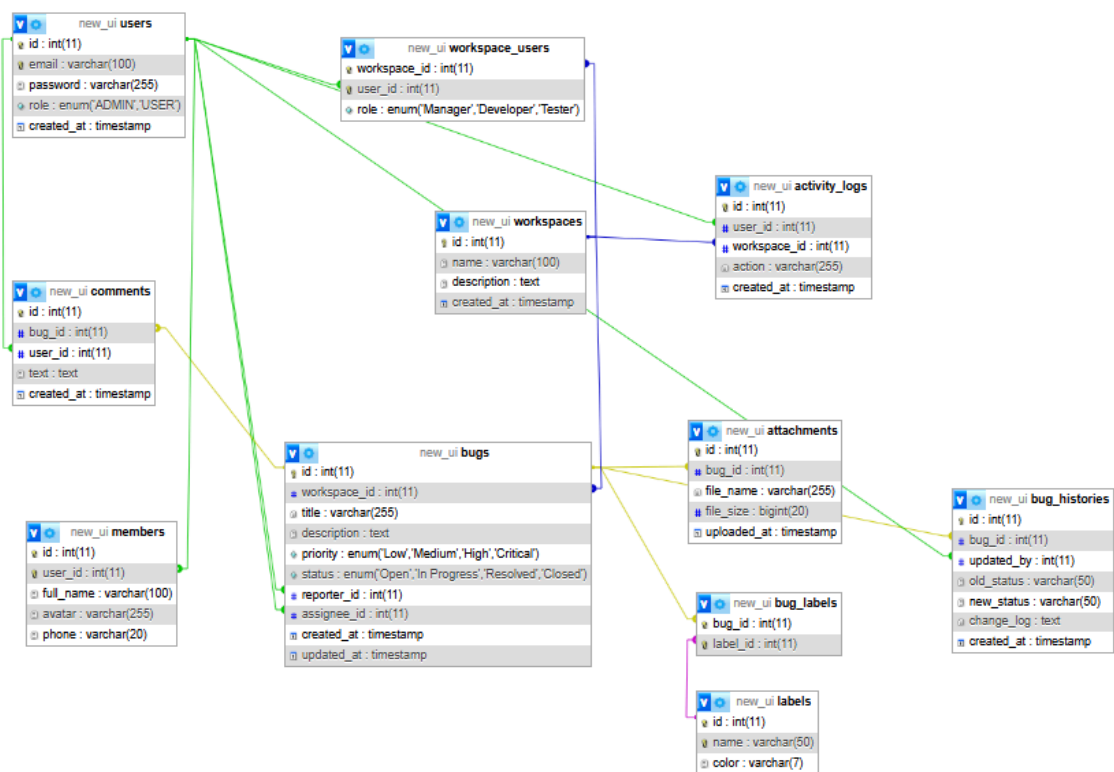
2. Chuẩn hóa lược đồ (Normalization)

Hệ thống được thiết kế đạt chuẩn **3NF**:

- **1NF:** Mọi giá trị thuộc tính là nguyên tử. Các danh sách (Labels, Users) đều được tách thành bảng trung gian riêng.
- **2NF:** Loại bỏ phụ thuộc hàm từng phần bằng cách tách thông tin Members khỏi Users.
- **3NF:** Loại bỏ phụ thuộc bắc cầu. Các thông tin lịch sử và tệp đính kèm được tách biệt hoàn toàn để bảo vệ dữ liệu gốc.

3. Lược đồ quan hệ (Relational Schema)

Lược đồ quan hệ mức vật lý thể hiện rõ các ràng buộc khóa ngoại (Foreign Key) giữa 11 bảng.



4. Từ điển dữ liệu (Data Dictionary)

Bảng users (Tài khoản người dùng)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh tài khoản
email	VARCHAR(100)	Unique, Not Null	Email đăng nhập

password	VARCHAR(255)	Not Null	Mật khẩu (đã mã hóa BCrypt)
role	ENUM	Not Null Default 'USER' ('ADMIN','USER')	Vai trò hệ thống (Quản trị toàn cục)
created_at	TIMESTAMP	Default CURRENT_TIME STAMP	Ngày tạo tài khoản

Bảng members (Hồ sơ thành viên)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh hồ sơ
user_id	INT	FK (users.id), Unique	Mã tài khoản liên kết
full_name	VARCHAR(100)	Not Null	Họ và tên thành viên
avatar	VARCHAR(255)	Nullable	Đường dẫn ảnh đại diện
phone	VARCHAR(20)	Nullable	Số điện thoại

Bảng workspaces (Không gian làm việc / Dự án)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh Workspace

name	VARCHAR(100)	Not Null	Tên không gian làm việc
description	TEXT	Nullable	Mô tả chi tiết dự án
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Ngày tạo dự án

Bảng workspace_users (Thành viên dự án)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
workspace_id	INT	PK(C), FK (workspaces.id)	Mã dự án tham gia
user_id	INT	PK(C), FK (users.id)	Mã thành viên tham gia
role	ENUM	Not Null ('Manager','Developer','Tester')	Vai trò thành viên trong dự án

Bảng bugs (Thông tin lỗi)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh lỗi
workspace_id	INT	FK (workspaces.id), Not Null	Lỗi thuộc dự án nào
title	VARCHAR(255)	Not Null	Tiêu đề lỗi

description	TEXT	Nullable	Mô tả chi tiết lỗi
priority	ENUM	Default 'Medium' ('Low','Medium','High','Critical')	Mức độ ưu tiên của lỗi
status	ENUM	Default 'Open' ('Open','In Progress','Resolved','Closed')	Trạng thái vòng đời lỗi
reporter_id	INT	FK (users.id), Not Null	Người phát hiện và báo cáo lỗi
assignee_id	INT	FK (users.id), Nullable	Người được phân công sửa lỗi
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Ngày tạo báo cáo lỗi
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Ngày cập nhật gần nhất

Bảng labels (Nhãn phân loại lỗi)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh nhãn

name	VARCHAR(50)	Not Null	Tên nhãn dán (Ví dụ: UI/UX, Backend)
color	VARCHAR(7)	Default '#6c757d'	Mã màu Hex hiển thị cho nhãn

Bảng bug_labels (Gắn nhãn cho lỗi)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
bug_id	INT	PK(C), FK (bugs.id)	Mã lỗi được gắn nhãn
label_id	INT	PK(C), FK (labels.id)	Mã nhãn dán được áp dụng

Bảng comments (Bình luận thảo luận)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh bình luận
bug_id	INT	FK (bugs.id), Not Null	Bình luận thuộc lỗi nào
user_id	INT	FK (users.id), Not Null	Thành viên viết bình luận
text	TEXT	Not Null	Nội dung thảo luận

created_at	TIMESTAMP	Default CURRENT_TIMES TAMP	Thời điểm bình luận
-------------------	-----------	----------------------------------	------------------------

Bảng attachments (Tập tin đính kèm)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh tệp tin
bug_id	INT	FK (bugs.id), Not Null	Tệp tin thuộc lỗi nào
file_name	VARCHAR(255)	Not Null	Tên tệp tin tải lên
file_size	BIGINT	Nullable	Dung lượng tệp tin tính bằng bytes
uploaded_at	TIMESTAMP	Default CURRENT_TIMES TAMP	Thời gian tải tệp tin lên

Bảng bug_histories (Lịch sử trạng thái lỗi)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã lịch sử thay đổi
bug_id	INT	FK (bugs.id), Not Null	Lịch sử của lỗi nào

updated_by	INT	FK (users.id), Not Null	Người thực hiện thay đổi
old_status	VARCHAR(50)	Nullable	Trạng thái cũ của lỗi
new_status	VARCHAR(50)	Nullable	Trạng thái mới của lỗi
change_log	TEXT	Nullable	Mô tả lý do hoặc nhật ký thay đổi
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Thời điểm cập nhật

Bảng activity_logs (Nhật ký hoạt động hệ thống)

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	PK, Auto Increment	Mã định danh nhật ký
user_id	INT	FK (users.id), Not Null	Người dùng thực hiện hành động
workspace_id	INT	FK (workspaces.id), Nullable	Dự án mà hành động diễn ra
action	VARCHAR(255)	Not Null	Mô tả chi tiết hành động

created_at	TIMESTAMP	Default CURRENT_TIMESTAMP	Thời điểm ghi nhận hệ thống
-------------------	-----------	------------------------------	-----------------------------------

IV. Thiết kế giao diện người dùng

1. Giao diện đăng nhập/đăng ký

Bug Tracker

Tạo tài khoản

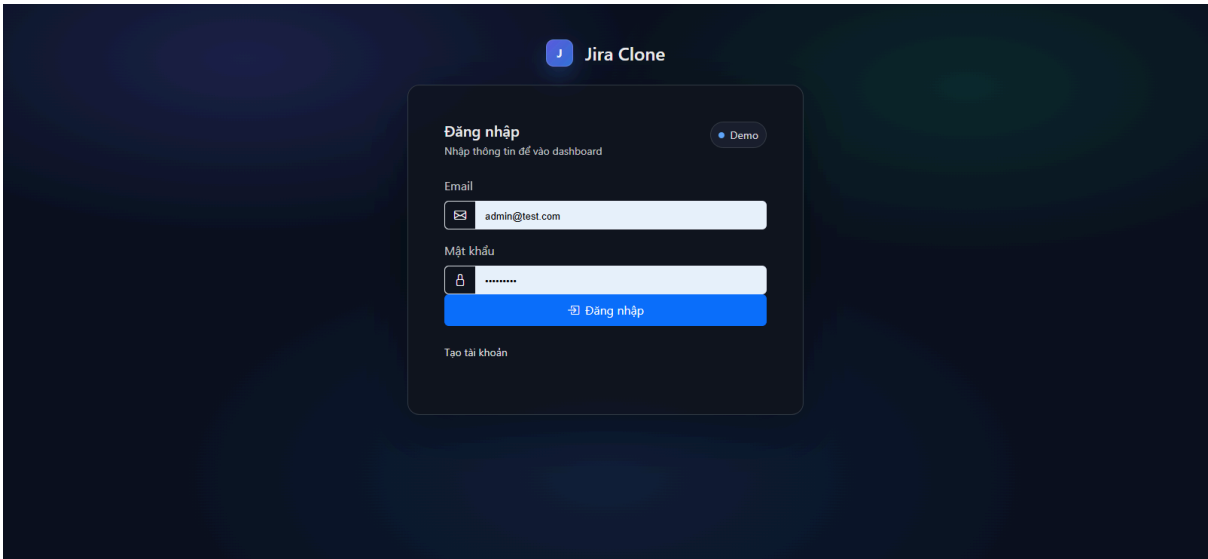
Tên:

Email:

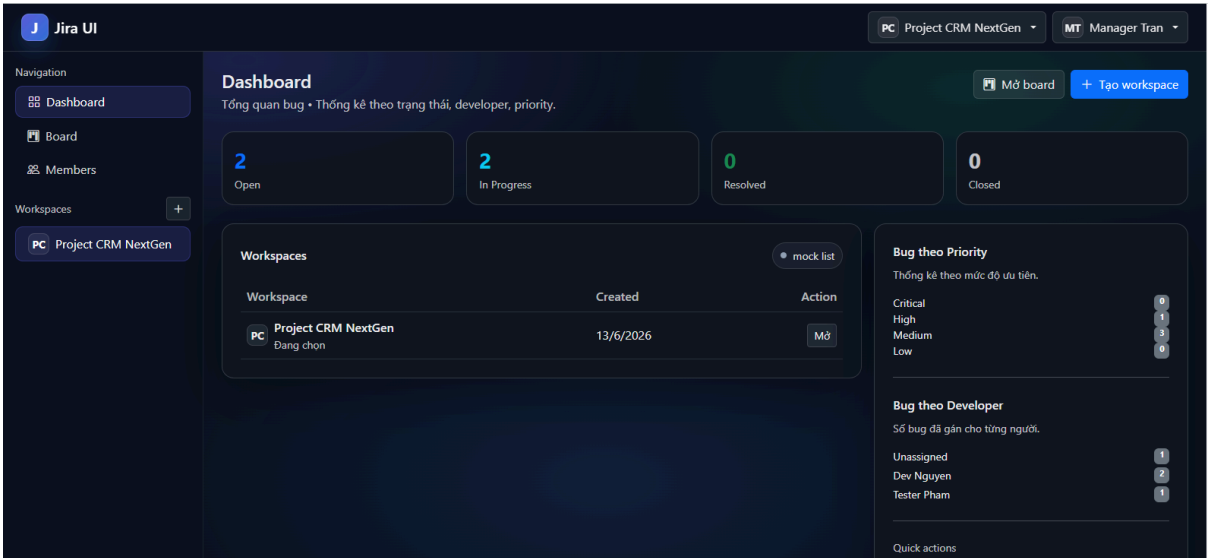
Mật khẩu:

[Tạo tài khoản](#)

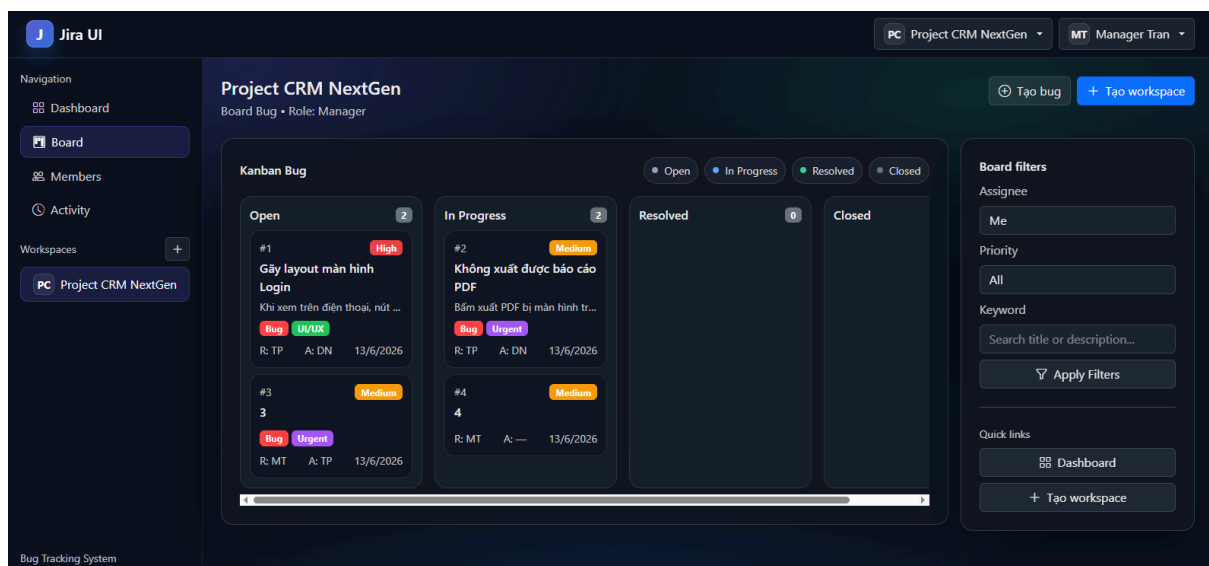
Đã có tài khoản? [Đăng nhập](#)



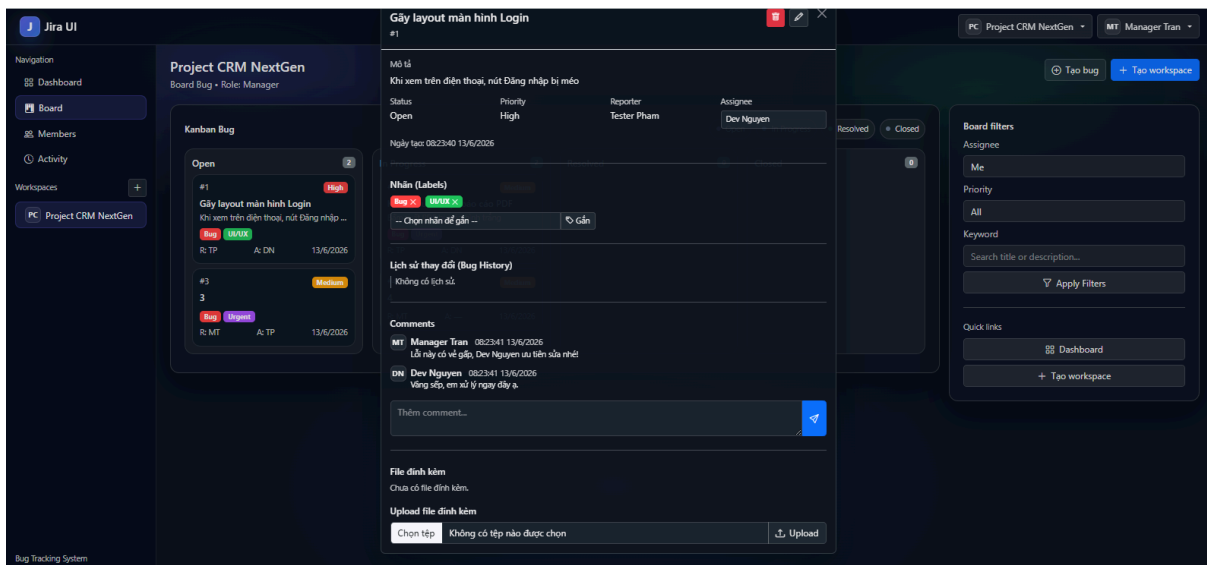
2. Giao diện trang chủ



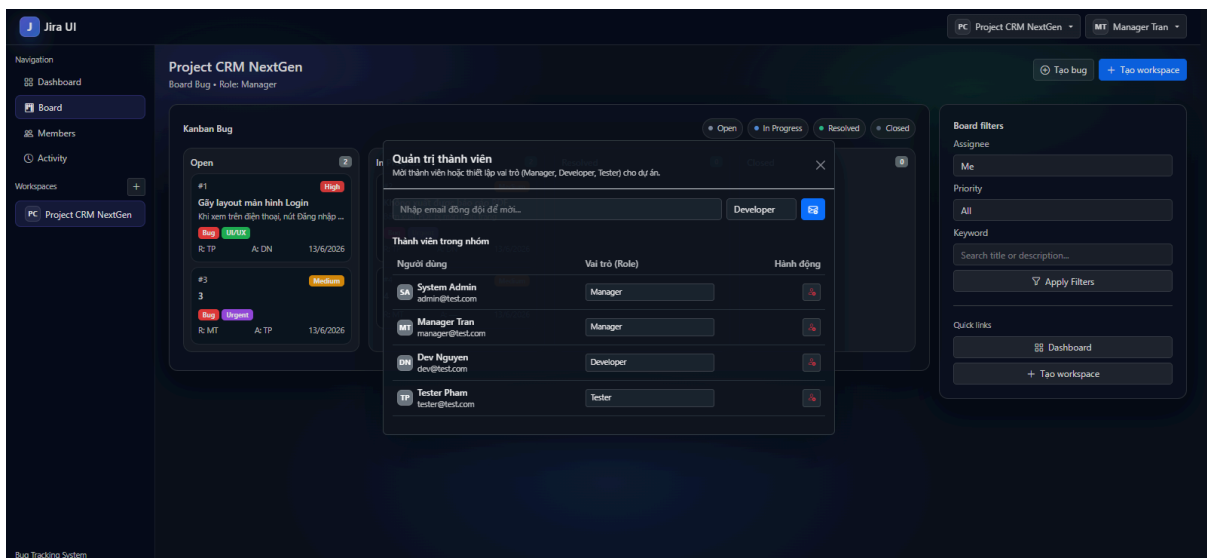
3. Giao diện danh sách Bug



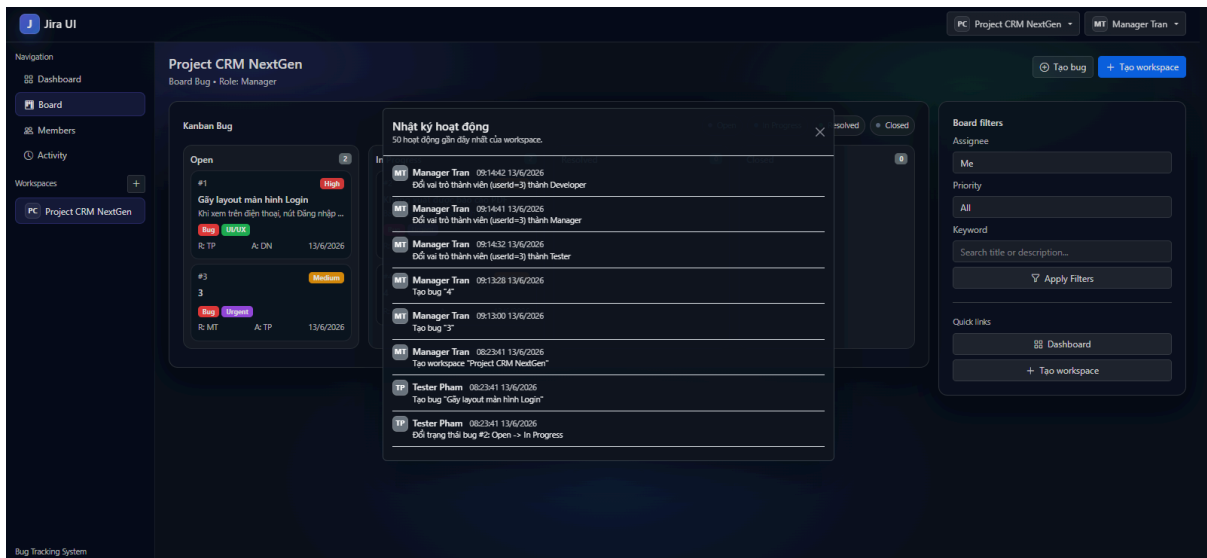
4. Giao diện chi tiết Bug



5. Giao diện quản trị thành viên



6. Giao diện nhật ký hoạt động



V: Cài đặt

1. Môi trường cài đặt và công nghệ sử dụng

Hệ thống được phát triển và kiểm thử trên môi trường với các yêu cầu tối thiểu như sau:

a) Yêu cầu môi trường:

- Java Development Kit (JDK): phiên bản 17 trở lên.
- Hệ quản trị cơ sở dữ liệu: MySQL/MariaDB (sử dụng XAMPP), chạy ở cổng 3307.
- Công cụ quản lý thư viện và build: Apache Maven (đã tích hợp sẵn qua Maven Wrapper - mvnw).
- Trình duyệt web hiện đại (Chrome, Edge, Firefox) để chạy giao diện người dùng.

b) Các bước cài đặt và khởi chạy:

- Bước 1 - Chuẩn bị cơ sở dữ liệu: Khởi động MySQL, tạo một cơ sở dữ liệu mới với tên new_ui, sau đó nhập (import) toàn bộ cấu trúc bảng và dữ liệu mẫu từ tệp database.sql vào cơ sở dữ liệu này.

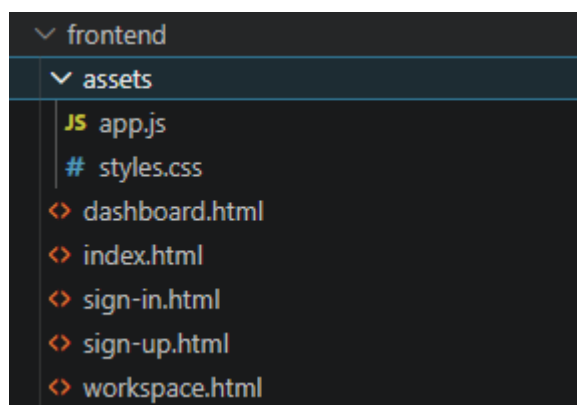
- Bước 2 - Cấu hình kết nối: Kiểm tra tệp application.properties trong thư mục backend để đảm bảo thông tin kết nối cơ sở dữ liệu (cổng, tên đăng nhập, mật khẩu) trùng khớp với máy chủ MySQL đang chạy.
- Bước 3 - Khởi chạy Backend: Mở cửa sổ dòng lệnh tại thư mục backend và thực thi lệnh `.\mvnw spring-boot:run`. Máy chủ sẽ khởi động tại địa chỉ `http://localhost:8080`. Tài liệu API có thể truy cập qua Swagger UI tại `http://localhost:8080/swagger-ui.html`.
- Bước 4 - Khởi chạy Frontend: Mở tệp `sign-in.html` trong thư mục frontend bằng trình duyệt (hoặc sử dụng tiện ích Live Server) để bắt đầu sử dụng hệ thống.

c) Tài khoản mẫu (dữ liệu seed):

- `admin@test.com`: tài khoản quản trị hệ thống (vai trò ADMIN), có toàn quyền trên mọi không gian làm việc.
- `manager@test.com`: tài khoản giữ vai trò Manager của dự án mẫu.
- `dev@test.com`: tài khoản giữ vai trò Developer.
- `tester@test.com`: tài khoản giữ vai trò Tester.

2. Tổ chức các lớp của chương trình

a) Frontend



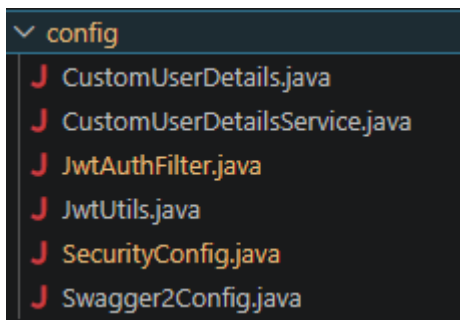
- `sign-in.html`: Màn hình đăng nhập hệ thống. Cho phép người dùng nhập Email và mật khẩu để xác thực tài khoản và nhận mã token JWT lưu vào LocalStorage.
- `sign-up.html`: Màn hình đăng ký tài khoản mới. Cho phép

người dùng nhập Họ tên, Email và Mật khẩu. Tài khoản mới mặc định mang vai trò hệ thống USER; vai trò trong dự án (Manager, Developer, Tester) được xác định sau khi tạo hoặc tham gia một Workspace.

- dashboard.html: Trang tổng quan quản trị. Hiển thị các khối thống kê số lượng lỗi theo trạng thái, theo mức độ ưu tiên, danh sách không gian làm việc (Workspace) tham gia và bảng quản lý thành viên.
- workspace.html: Bảng điều khiển Kanban chi tiết. Hiển thị danh sách thẻ lỗi theo 4 cột trạng thái (Open, In Progress, Resolved, Closed), tích hợp các hộp thoại xem chi tiết lỗi, bình luận, đính kèm tệp và cập nhật thông tin lỗi.

b) Backend

Package config:

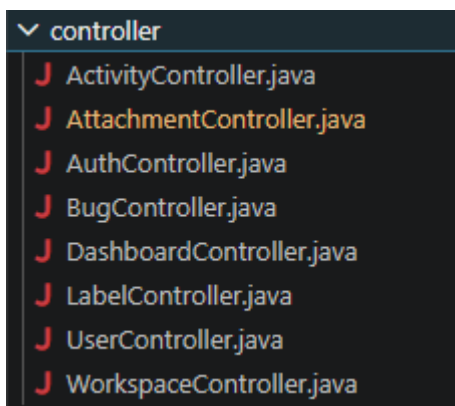


- SecurityConfig.java: Cấu hình bảo mật hệ thống. Định nghĩa các endpoint công khai và các endpoint bắt buộc xác thực, cấu hình CORS hỗ trợ gọi API chéo nguồn, và thiết lập chuỗi bộ lọc bảo mật.
- JwtAuthFilter.java: Bộ lọc kiểm soát mọi yêu cầu HTTP gửi đến. Trích xuất mã JWT từ Header Authorization, xác thực tính hợp lệ của token và nạp thông tin người dùng vào khả năng bảo mật của Spring Security.
- JwtUtils.java: Lớp tiện ích xử lý Token JWT. Thực hiện mã hóa tạo token mới từ thông tin email người dùng, giải mã token và kiểm tra tính hợp hạn sử dụng với khóa ký cố định cấu hình từ application.properties.
- CustomUserDetails.java: Lớp bọc thông tin người dùng đăng

nhập. Kế thừa UserDetails của Spring Security nhằm lưu trữ thông tin tài khoản và cấp các quyền tương ứng của người dùng.

- CustomUserDetailsService.java: Dịch vụ tìm kiếm tài khoản. Thực hiện truy vấn thông tin tài khoản từ cơ sở dữ liệu qua email phục vụ cho quá trình đăng nhập và xác thực của Spring Security.
- Swagger2Config.java: Cấu hình tự động sinh tài liệu API trực quan dạng Swagger UI để lập trình viên dễ dàng thử nghiệm API.

Package controller:



- AuthController.java: Tiếp nhận và điều phối các yêu cầu đăng nhập (/api/auth/login) và đăng ký tài khoản (/api/auth/register).
- UserController.java: Cung cấp các API quản lý thông tin tài khoản người dùng (/users), cập nhật hồ sơ cá nhân và tải lên ảnh đại diện cá nhân.
- WorkspaceController.java: Cung cấp các API quản trị dự án (/api/workspaces), bao gồm tạo mới, xóa dự án, mời thành viên qua email và điều chỉnh vai trò thành viên.
- BugController.java: Cung cấp các API xử lý nghiệp vụ quản lý lỗi (/api/bugs), tạo bình luận dưới thẻ lỗi, truy vấn nhật ký thay đổi của một lỗi.
- AttachmentController.java: Cung cấp các API quản lý tệp tin đính kèm của lỗi, cho phép upload tệp minh chứng lỗi lên máy chủ và tải về (download).

- DashboardController.java: Cung cấp các API thống kê số lượng lỗi trong không gian làm việc phục vụ vẽ biểu đồ Dashboard.
- LabelController.java: Cung cấp các API quản lý nhãn dán (Label) phân loại lỗi trong từng dự án và gắn/gỡ nhãn cho lỗi.
- ActivityController.java: Cung cấp các API truy vấn nhật ký hoạt động (Activity Log) của người dùng trong dự án.

Package entity

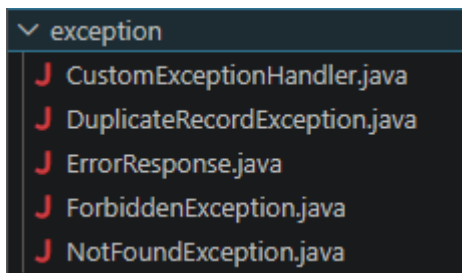


- User.java: Thực thể ánh xạ với bảng users trong cơ sở dữ liệu để quản lý tài khoản đăng nhập, mật khẩu đã mã hóa và vai trò hệ thống (role: ADMIN/USER) dùng để phân quyền quản trị toàn cục.
- Member.java: Thực thể ánh xạ với bảng members, lưu hồ sơ cá nhân của người dùng (họ tên, ảnh đại diện, số điện thoại).
- Workspace.java: Thực thể ánh xạ với bảng workspaces quản lý thông tin các không gian làm việc của dự án.
- WorkspaceUser.java / WorkspaceUserId.java: Thực thể ánh xạ bảng liên kết workspace_users để lưu thông tin vai trò của thành viên trong từng dự án cụ thể.
- Bug.java: Thực thể ánh xạ bảng bugs lưu thông tin mô tả chi

tiết lỗi, mức độ ưu tiên và trạng thái lỗi.

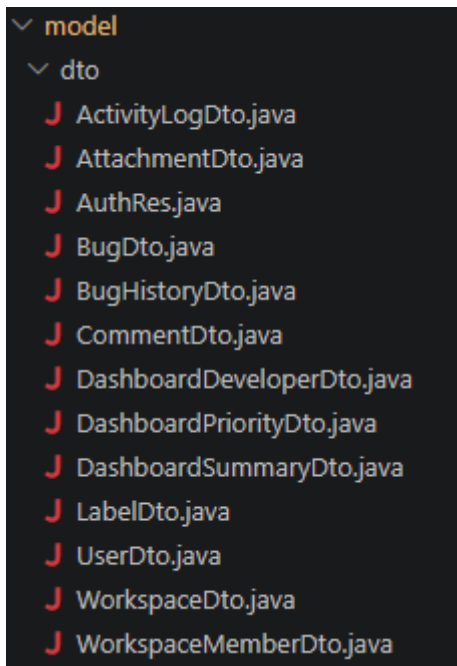
- Comment.java: Thực thể ánh xạ bảng comments quản lý các đoạn thảo luận về lỗi dưới dạng văn bản.
- Attachment.java: Thực thể ánh xạ bảng attachments quản lý đường dẫn lưu trữ, tên file và dung lượng file đính kèm của lỗi.
- BugHistory.java: Thực thể ánh xạ bảng bug_histories để lưu vết mọi lịch sử biến động trạng thái và người phụ trách lỗi.
- Label.java: Thực thể ánh xạ bảng labels để định nghĩa nhãn dán phân loại lỗi.
- BugLabel.java / BugLabelId.java: Thực thể ánh xạ bảng trung gian liên kết giữa lỗi và nhãn dán.
- ActivityLog.java: Thực thể ánh xạ bảng activity_logs lưu nhật ký hoạt động chung trên hệ thống.

Package Exception



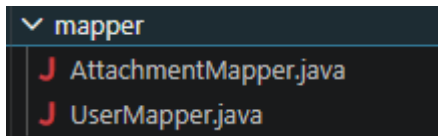
- CustomExceptionHandler.java: Lớp bắt lỗi tập trung (Global Exception Handler) xử lý toàn bộ các Exception phát sinh trong Runtime để trả về Error Response chuẩn hóa cho Client.
- NotFoundException.java: Ngoại lệ ném ra khi không tìm thấy thực thể tương ứng trong database.
- DuplicateRecordException.java: Ngoại lệ ném ra khi gặp lỗi trùng lặp dữ liệu (ví dụ: đăng ký trùng email).
- ForbiddenException.java: Ngoại lệ ném ra khi người dùng cố tình thực hiện các thao tác vượt quá quyền hạn của vai trò.
- ErrorResponse.java: Cấu trúc dữ liệu lỗi tiêu chuẩn (mã lỗi, thông điệp, mốc thời gian) trả về cho Client.

Package model/dto (Đối tượng truyền dữ liệu - DTO):



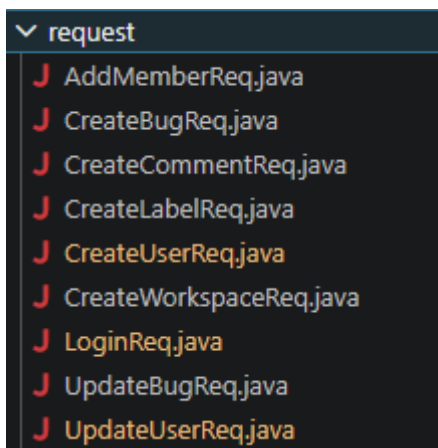
- UserDto.java: Đối tượng đóng gói thông tin người dùng (ID, tên, email, ảnh đại diện, vai trò) trả về cho Client.
- WorkspaceDto.java: Chứa thông tin Workspace và danh sách các thành viên cùng vai trò cụ thể của họ trong dự án.
- WorkspaceMemberDto.java: Thông tin cá nhân của 1 thành viên
- BugDto.java: Đối tượng truyền tải chi tiết thông tin lỗi, mức độ ưu tiên, trạng thái, người tạo và người chịu trách nhiệm sửa.
- CommentDto.java: Chứa nội dung bình luận và thông tin người đăng bình luận nhằm hiển thị dưới dạng thảo luận nhóm.
- AttachmentDto.java: Chứa thông tin tệp tin đính kèm để hiển thị và tải về.
- BugHistoryDto.java: Đóng gói dữ liệu lịch sử thay đổi để hiển thị dưới dạng timeline hoạt động của lỗi.
- AuthRes.java: Chứa thông tin phản hồi sau khi xác thực thành công (Token JWT và thông tin người dùng).
- DashboardSummaryDto.java / DashboardPriorityDto.java / DashboardDeveloperDto.java: Đóng gói các chỉ số thống kê phục vụ vẽ biểu đồ trên Dashboard.
- LabelDto.java: chứa thông tin về nhãn dán.

Package model/mapper:



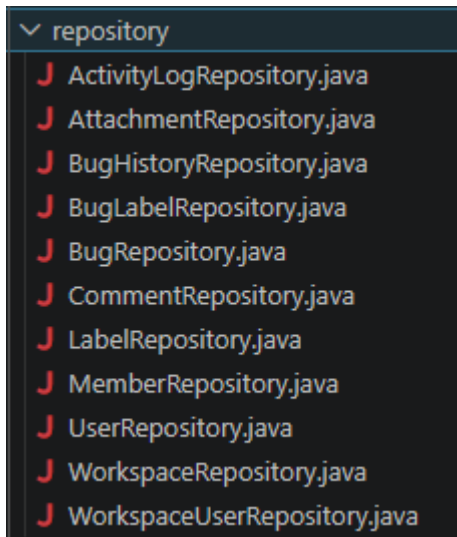
- UserMapper.java: Lớp chuyển đổi qua lại giữa thực thể User và các đối tượng UserDto, CreateUserReq nhằm tách biệt dữ liệu nội bộ với dữ liệu trao đổi với client.
- AttachmentMapper.java: Lớp chuyển đổi giữa thực thể Attachment và AttachmentDto khi hiển thị thông tin tệp đính kèm.

Package model/request:



- Các lớp tiếp nhận và kiểm tra dữ liệu đầu vào (request body) từ client trước khi xử lý, gồm: CreateUserReq, LoginReq (đăng ký, đăng nhập), CreateWorkspaceReq, AddMemberReq (tạo dự án, mời thành viên), CreateBugReq, UpdateBugReq (tạo và cập nhật lỗi), CreateCommentReq, CreateLabelReq và UpdateUserReq.

Package repository (Tầng truy xuất dữ liệu):



- Tập hợp các interface kế thừa JpaRepository của Spring Data JPA, cung cấp sẵn các thao tác truy vấn cơ sở dữ liệu (thêm, sửa, xóa, tìm kiếm) cho từng thực thể tương ứng: UserRepository, MemberRepository, WorkspaceRepository, WorkspaceUserRepository, BugRepository, BugHistoryRepository, CommentRepository, AttachmentRepository, LabelRepository, BugLabelRepository và ActivityLogRepository.

Package service (Tầng nghiệp vụ):



- Chứa toàn bộ logic nghiệp vụ của hệ thống, được thiết kế theo

cặp interface và lớp cài đặt (Service / ServiceImpl) để tách biệt phần khai báo với phần hiện thực, giúp mã nguồn dễ bảo trì và mở rộng.

- UserService / UserServiceImpl: Xử lý đăng ký, quản lý tài khoản và hồ sơ người dùng.
- WorkspaceService / WorkspaceServiceImpl: Xử lý tạo, xóa dự án, quản lý thành viên và kiểm soát phân quyền theo vai trò.
- BugService / BugServiceImpl: Xử lý vòng đời của lỗi, gán người xử lý, ghi lịch sử thay đổi và bình luận.
- LabelService / LabelServiceImpl: Quản lý nhãn dán và việc gán nhãn cho lỗi.
- AttachmentService / AttachmentServiceImpl: Xử lý lưu trữ và truy xuất tệp đính kèm.
- ActivityService / ActivityServiceImpl: Ghi nhận và truy vấn nhật ký hoạt động của dự án.
- DashboardService / DashboardServiceImpl: Tổng hợp số liệu thống kê phục vụ biểu đồ trên Dashboard.

VI: Kết Luận và hướng phát triển

Sau quá trình nghiên cứu, thiết kế và triển khai thực nghiệm đề tài "**Xây dựng hệ thống quản lý Bug**", nhóm thực hiện đề tài đã hoàn thành toàn bộ các mục tiêu đặt ra ban đầu, đồng thời thu được các kết quả cụ thể như sau:

1. Kết quả đạt được

Về mặt lý thuyết

- **Quản trị dự án:** Hệ thống hóa thành công quy trình quản trị dự án phần mềm theo mô hình Agile/Kanban và hiểu rõ quy trình vòng đời của lỗi phần mềm (Bug Life Cycle).
- **Thiết kế hệ thống:** Nắm vững và áp dụng thành thạo phương pháp phân tích thiết kế hệ thống thông tin theo chuẩn 3NF (Third Normal Form) cùng kiến trúc phần mềm Client-Server (Khách - Chủ).

Về mặt kỹ thuật và triển khai

- **Phân hệ Backend:** Xây dựng thành công hệ thống RESTful API hoàn chỉnh bằng ngôn ngữ Java (Spring Boot 3.5). Áp dụng giải pháp bảo mật toàn diện với Spring Security kết hợp mã hóa băm mật khẩu BCrypt và cơ chế xác thực không trạng thái qua mã thông báo JWT (JSON Web Token). Đồng thời, tự động hóa tài liệu hóa API thông qua công cụ Swagger UI.
- **Cơ sở dữ liệu (Database):** Thiết kế và tối ưu hóa hệ cơ sở dữ liệu quan hệ MySQL gồm 11 bảng đạt chuẩn hóa 3NF; thiết lập đầy đủ các ràng buộc khóa ngoại và logic lan truyền (ON DELETE CASCADE / ON UPDATE CASCADE) nhằm bảo đảm tính toàn vẹn dữ liệu tối đa.
- **Phân hệ Frontend:** Hoàn thiện giao diện ứng dụng Web hiện đại, hỗ trợ chế độ nền tối (Dark Mode) và khả năng hiển thị co giãn tương thích (Responsive) trên cả thiết bị di động lẫn máy tính để bàn nhờ sử dụng HTML5, CSS3, Bootstrap 5 và Vanilla JavaScript. Thực hiện tối ưu cơ chế gọi API bất đồng bộ (Asynchronous) và xử lý luồng tải tập tin mượt mà.

2. Những điểm còn hạn chế

Mặc dù hệ thống đã vận hành ổn định và đáp ứng đầy đủ các yêu cầu chức năng cốt lõi của một công cụ Bug Tracking, dự án vẫn tồn tại một số điểm hạn chế cần khắc phục do giới hạn về mặt thời gian:

- **Phương thức lưu trữ tệp tin:** Hiện tại hệ thống đang lưu trữ trực

tiếp các tệp đính kèm (Attachments) vào ổ đĩa vật lý của máy chủ thông qua đường dẫn cục bộ (System.getProperty("user.home")). Giải pháp này chưa tối ưu khi triển khai trên môi trường phân tán hoặc đám mây (Cloud) do bị giới hạn bởi dung lượng vật lý và tính sẵn sàng của dữ liệu.

- **Trải nghiệm người dùng (UX) trên bảng Kanban:** Bảng Kanban hiện tại mới chỉ hỗ trợ cập nhật trạng thái lỗi thông qua hộp chọn (Select Box) trong cửa sổ chi tiết, chưa tích hợp được tính năng kéo - thả (Drag & Drop) trực tiếp các thẻ lỗi giữa các cột trạng thái.
- **Tính năng tương tác thời gian thực:** Hệ thống chưa có cơ chế tự động đẩy thông báo (Real-time Notifications) khi có biến động dữ liệu (ví dụ: khi thành viên được gán lỗi mới, hoặc khi có bình luận mới). Người dùng vẫn phải thực hiện tải lại trang (Refresh) để cập nhật trạng thái mới nhất.
- **Báo cáo và thống kê:** Màn hình tổng quan (Dashboard) mới dừng lại ở mức cơ bản với các thẻ số liệu và biểu đồ tĩnh, chưa cung cấp được các biểu đồ tiến độ chuyên sâu cho quản trị dự án như Biểu đồ tiến độ dự án (Burndown Chart) hay Biểu đồ luồng tích lũy (Cumulative Flow Diagram).

3. Hướng phát triển của đề tài

Để hệ thống trở nên hoàn thiện, nâng cao tính bảo mật và có khả năng ứng dụng thực tiễn cao tại các doanh nghiệp phần mềm, nhóm định hướng các bước phát triển tiếp theo như sau:

- **Tích hợp dịch vụ lưu trữ đám mây:** Thay thế phương thức lưu trữ file trên ổ cứng cục bộ bằng các dịch vụ lưu trữ đối tượng chuyên dụng như Amazon S3, Cloudinary hoặc Google Cloud Storage nhằm tối ưu hiệu suất và tăng tính an toàn cho tài nguyên.
- **Nâng cấp giao diện kéo thả trực quan:** Tích hợp HTML5 Drag and Drop API hoặc các thư viện JavaScript chuyên dụng nhằm tối ưu hóa bảng Kanban, cho phép người dùng chuyển trạng thái bug một cách nhanh chóng.

- **Xây dựng tính năng thông báo thời gian thực (Real-time):** Triển khai giao thức kết nối hai chiều WebSockets hoặc Server-Sent Events (SSE) để đẩy thông báo tức thời tới màn hình người dùng ngay khi có tương tác mới phát sinh.
- **Mở rộng bộ công cụ thống kê Dashboard:** Tích hợp các thư viện biểu đồ động như Chart.js hoặc D3.js để trực quan hóa hiệu suất làm việc của lập trình viên, tốc độ giải quyết lỗi (Resolution Rate) và biểu đồ Burndown Chart.
- **Bổ sung kiểm thử tự động (Automated Testing):** Xây dựng hệ thống các bài kiểm thử đơn vị (Unit Test) và kiểm thử tích hợp (Integration Test) bằng JUnit 5 và Mockito ở phía Backend, bảo đảm tính ổn định và toàn vẹn của mã nguồn trước khi nâng cấp các tính năng mới.